HotelOS (e-hotel)

Спецификация REST API для бэкенда

Версия 1.0 · 29.10.2026

Что фронтенд (Angular 22) ожидает от сервера: эндпоинты, модели, ошибки, роли, бизнес-правила и сценарии. К документу приложен **openapi.yaml** с теми же данными.

Содержание

1. Как пользоваться документом **3**

Если вы работаете через Claude Code . . . 3

2. Что делает система **3**

3. Общие правила API **4**

4. Авторизация и роли **4**

4.1. Токены . . . 4

4.2. Роли и доступ к разделам . . . 4

5. Ошибки **5**

5.1. Коды ошибок . . . 5

6. Основные сценарии **6**

7. Эндпоинты **8**

7.1. Авторизация . . . 9

7.2. Дашборд . . . 10

7.3. Номера . . . 11

7.4. Книга гостей . . . 13

7.5. Гости (архив) . . . 14

7.6. Чёрный список . . . 14

7.7. Фискальный чек . . . 15

7.8. Расходы . . . 16

7.9. Сотрудники и роли . . . 17

7.10. Настройки . . . 18

7.11. Комментарии и QR-коды . . . 19

8. Модели данных **21**

8.1. Модели . . . 21

8.2. Перечисления . . . 30

9. Журнал действий **30**

10. Подключение фронта (для справки) **31**

11. Рекомендуемая схема БД **31**

12. Чек-лист приёмки **32**

1. Как пользоваться документом

Фронтенд HotelOS готов и работает на моках: каждый экран берёт данные из сервисов `src/app/core/data/*.service.ts`, которые сейчас возвращают тестовые данные с задержкой. Формы данных в этих сервисах уже совпадают с будущим API. Задача бэкенда — реализовать эндпоинты из раздела 7 с моделями из раздела 8 и правилами из разделов 3–6; после этого фронт заменит тела методов на HttpClient без изменения экранов.

Разделы 3–6 — общие правила (формат, авторизация, роли, ошибки, сценарии), их стоит прочитать целиком. Раздел 7 — справочник по эндпоинтам, раздел 8 — модели. В конце — рекомендуемая схема БД и чек-лист приёмки.

Если вы работаете через Claude Code

Положите оба файла в репозиторий бэкенда (например, в папку docs/) и дайте Claude Code задание со ссылкой на них. openapi.yaml — машиночитаемая часть (пути, схемы, коды ответов), PDF — бизнес-правила, которых нет в схеме. Пример первого запроса:

```
Прочитай docs/hotelos-api.pdf и docs/openapi.yaml. Это спецификация API для
фронтенда HotelOS. Предложи структуру проекта и схему БД, затем реализуй
по шагам: 1) авторизация и роли, 2) номера и категории, 3) книга гостей,
выселение и архив, 4) чёрный список и фискальный чек, 5) расходы, пользователи,
настройки, журнал, 6) дашборд, 7) комментарии и QR. После каждого шага —
тесты на бизнес-правила из PDF (разделы 3–7).
```

Главное. Имена полей, значения перечислений и коды ошибок фронт использует как есть — их нельзя переименовывать. Всё остальное (стек, БД, структура кода) — на усмотрение бэкенда.

2. Что делает система

HotelOS — система управления небольшим отелем/хостелом в Узбекистане. Интерфейс на трёх языках (ru, uz-Latn, uz-Cyrl), десктоп и мобильная версия. Один отель на установку.

Раздел интерфейса	Что делает	Эндпоинты
Вход	Логин и пароль, «запомнить меня»	`/auth/*`
Дашборд	Загрузка, заезды/выезды сегодня, выручка и расходы по дням, номерной фонд	`/dashboard`
Номера	Карточки номеров, гости в номере, заселение, подселение, продление, выселение с суммой, фискальным чеком и ЧС, уборка, QR-код номера	`/rooms/*, /book, /fiscal/receipts`
Книга	Все заселённые гости: добавить (регистрация + заселение), изменить, в ЧС, полная карточка	`/book/*`
Гости	Архив выехавших гостей, в ЧС, полная карточка	`/guests/*`
Чёрный список	Список, добавить вручную (по документу с поиском в архиве), убрать	`/blacklist/*`
Расходы	CRUD расходов, итоги по фильтру	`/expenses/*, /staff`
Комментарии	Отзывы гостей по QR-коду в номере, публичная страница /c/{room}	`/comments/*, /qr/*`
Пользователи	Сотрудники, роли, блокировка, сброс пароля, матрица прав	`/users/*, /roles/*`
Настройки	Данные отеля, категории и тарифы, журнал действий	`/settings/*, /categories/*, /audit-log`

3. Общие правила API

Тема	Правило
Базовый путь	`/api/v1`. Dev-сервер фронта проксирует /api на бэкенд (proxy.conf.json), поэтому отдельный CORS для разработки не нужен. В проде — тот же домен или CORS для домена фронта.
Формат	JSON, UTF-8. Имена полей — camelCase ровно как в моделях раздела 8.
Дата	`IsoDate` = строка `YYYY-MM-DD`.
Дата и время	`IsoDateTime` = строка `YYYY-MM-DDTHH:mm` — **местное время отеля (Asia/Tashkent, UTC+5)**, без секунд и без зоны. Сравнивать как строки нельзя только при смене формата.
«Сегодня»	Берётся на сервере в часовом поясе отеля. Сейчас фронт использует фиксированную дату 2026-10-26 (токен APP_TODAY) — после подключения API её даст сервер.
Деньги	Целые числа в сумах (UZS), без тийинов и без дробей. Форматирование «4 800 000» — на фронте.
Страны	Коды ISO 3166-1 alpha-2: UZ, RU, KZ, GB, DE, ES, US, CN, JP… Названия переводит фронт.
Документ гостя	Хранить как прислали («AB 1234567»). Для сравнения и поиска — нормализовать: убрать пробелы, верхний регистр (`AB1234567`). Уникальность проживания, ЧС, поиск — по нормализованному.
Номер комнаты	Строка («206», «12A») и он же идентификатор в URL: `/rooms/206`.
Списки	Возвращать полный массив без пагинации: фронт сам сортирует, фильтрует по столбцам и листает страницы (AG Grid). Если данных станет много (тысячи строк) — договориться о серверной пагинации.
Перечисления	Только значения из раздела 8.2 (строки в нижнем регистре / camelCase).
Пустые значения	Необязательные текстовые поля — пустая строка "", не null. Даты, которых нет, — null (если в модели указано |null).
Ответ на удаление	204 без тела. Создание — 201 с созданной записью. Изменение — 200 с записью целиком.
Журнал действий	Пишет сам бэкенд при перечисленных в разделе 9 действиях — фронт ничего не отправляет.

4. Авторизация и роли

4.1. Токены

- Вход: `POST /auth/login` → accessToken (JWT, ~15 минут) + refreshToken.
- Все запросы, кроме публичных, — с заголовком `Authorization: Bearer <accessToken>`.
- Истёк accessToken → 401 `auth.unauthorized` → фронт вызывает `POST /auth/refresh` и повторяет запрос; refresh не прошёл — выход на экран входа.
- В JWT достаточно: id сотрудника и роль. Блокировка сотрудника должна действовать сразу (проверять active или отзывать токены).
- Публичные эндпоинты (без токена): `POST /auth/login`, `POST /auth/refresh`, `GET /qr/{room}`, `POST /comments` — их открывает гость с телефона по QR-коду.

4.2. Роли и доступ к разделам

Права задаются по разделам. **2** — полный доступ (чтение и изменение), **1** — только чтение (GET), **0** — нет доступа (403 auth.forbidden). Эта же матрица отдаётся в `GET /roles/permissions` и показывается в «Пользователях» только для чтения.

Раздел	Что входит	owner	admin	reception	housekeeper	accountant
`dashboard`	Дашборд и выручка	полный	полный	—	—	полный

Раздел	Что входит	owner	admin	reception	housekeeper	accountant
registration	Регистрация и выселение гостей	полный	полный	полный	—	—
rooms	Номера и статусы уборки	полный	полный	полный	полный	—
book	Книга гостей и заселение	полный	полный	полный	—	чтение
blacklist	Чёрный список	полный	полный	чтение	—	—
archive	Архив гостей	полный	полный	чтение	—	чтение
expenses	Расходы	полный	полный	—	—	полный
admin	Пользователи и настройки	полный	полный	—	—	—

- В разделе 7 у каждого эндпоинта указан доступ: например, `book ≥ 1` — раздел book с уровнем 1 или 2; `admin` — раздел admin (владелец и администратор).
- Фронт прячет недоступные разделы в меню, но проверять права обязан сервер.
- Владелец и администратор: owner, admin. Роль owner нельзя снять с последнего владельца (рекомендация).

5. Ошибки

Любая ошибка — JSON одного вида. Фронт берёт **code** и показывает переведённый текст `errors.<code>` с подстановками из **params**. message — для логов, пользователю не показывается.

```
HTTP/1.1 409 Conflict
{
  "error": {
    "code": "rooms.exists",
    "message": "Room 206 already exists",
    "params": { "number": "206" }
  }
}

HTTP/1.1 422 Unprocessable Entity
{
  "error": {
    "code": "validation.failed",
    "message": "Validation failed",
    "fields": { "lastName": "required", "amount": "min" }
  }
}
```

HTTP-коды: 400 — неверный JSON; 401 — нет/истёк токен; 403 — нет прав; 404 — нет записи; 409 — конфликт с текущим состоянием (дубликат, нет мест, статус); 422 — ошибка в данных; 500 — всё прочее.

5.1. Коды ошибок

Колонка «Перевод»: **есть** — фронт уже показывает текст; **новый** — код нужен, фронт добавит перевод при подключении.

Код	HTTP	Когда / текст для пользователя	Перевод
auth.invalid	401	Неверный логин или пароль	есть
auth.blocked	403	Учётная запись заблокирована	есть
auth.unauthorized	401	Нет токена или он истёк — фронт уходит на вход	новый
auth.forbidden	403	Роли не хватает прав на раздел	новый
common.notFound	404	Запись не найдена	есть
common.unknown	500	Что-то пошло не так (любая непредвиденная ошибка)	есть

Код	HTTP	Когда / текст для пользователя	Перевод
validation.failed	422	Ошибка в данных; подробности — в error.fields	новый
validation.dates	422	Дата выезда должна быть позже даты заезда	новый
rooms.exists	409	Номер с таким числом уже есть (params.number)	есть
rooms.full	409	В номере не хватает мест (params.room, params.free)	новый
rooms.unavailable	409	Номер на уборке/ремонте — заселять нельзя (params.room)	новый
rooms.invalidStatus	409	Действие недоступно в текущем статусе номера	новый
rooms.hasGuests	409	Нельзя изменить — в номере гости (например, уменьшить места)	новый
guests.notFound	404	Гость не найден в архиве (params.doc)	есть
guests.duplicateDoc	422	Один документ у нескольких гостей в одной регистрации (params.doc)	новый
guests.alreadyIn	409	Гость с этим документом уже проживает (params.doc, params.room)	новый
blacklist.exists	409	Документ уже в чёрном списке (params.doc)	есть
blacklist.found	409	Гость в чёрном списке, нужен confirmBlacklisted (params.docs)	новый
settings.innInvalid	422	ИНН должен состоять из 9 цифр	есть
settings.categoryExists	409	Категория с таким названием уже есть (params.name)	есть
settings.categoryInUse	409	Категория используется номерами — удалить нельзя	новый
users.loginTaken	409	Логин занят (params.login)	есть
users.self	403	Нельзя заблокировать/удалить себя	новый
comments.qrDisabled	403	Для номера не включён QR-код (публичная форма)	новый

6. Основные сценарии

Как экраны используют API — по шагам. Это главные бизнес-правила; детали — у эндпоинтов в разделе 7.

Регистрация и заселение (одного или нескольких гостей)

1. Ресепшн вводит документ → «лупа»: GET /guests/lookup?doc= — форма заполняется, если гость уже был.
2. Фронт проверяет ЧС: GET /blacklist/check?doc=… Если есть совпадения — модалка с причиной («Отмена» / «Всё равно продолжить»).
3. Номер выбирается из GET /rooms/available (свободные и с местами).
4. «+» в модалке добавляет таб следующего гостя; все уходят одним запросом POST /book { registrations: [...] }.
5. Если гость в ЧС и сотрудник согласился — confirmBlacklisted: true. Без него бэкенд вернёт 409 blacklist.found.
6. Ответ 201 → фронт обновляет «Номера» и «Книгу». Номер становится busy.

Подселение в занятый номер

1. В карточке номера со свободными местами (beds > guests.length) — кнопка «Добавить гостя».
2. Тот же POST /book с room = этот номер. Бэкенд проверяет места (rooms.full).

Изменение данных гостя

1. Книга → «Изменить» → PUT /book/{id} с полной регистрацией.
2. Смена номера = переезд (проверка мест в новом номере).

Продление

1. Карточка номера → «Продлить» → POST /rooms/{n}/extend { checkout } — для всех гостей номера.

Выселение (закрытие клиента)

1. Карточка номера → «Выселить» → окно: выбор гостей (если их несколько), итоговая сумма, фискальный чек, галочка «В чёрный список» с причиной.
2. «Сформировать фискальный чек» → POST /fiscal/receipts → номер; фронт сам скачивает Word по шаблону.
3. «Выселить» → POST /rooms/{n}/checkout { docs, amount, receipt, blacklistReason }.
4. Бэкенд: гости → архив, при необходимости → ЧС, номер → clean (если уехали все), запись для выручки, журнал.

Уборка

1. После выселения номер clean. Горничная/ресепшн → «Отметить как готов» → POST /rooms/{n}/ready → free.

Модель данных гостя и номера

- **Книга (BookEntry)** — источник правды о проживающих. Room.guests — проекция книги (краткие данные).
- **Номер busy** ⇔ в нём есть хотя бы один гость книги. Мест: beds, свободно: beds − guests.length.
- Выехавшие гости — только в архиве (ArchivedGuest). Из книги гость уходит только через выселение.
- **Профиль гостя** по документу (последние известные данные) — для «лупы» (GET /guests/lookup).
- **Чёрный список** не мешает хранить гостя: это только предупреждение при заселении (решает сотрудник).

7. Эндпоинты

Все пути — от `/api/v1`. Сводная таблица, затем подробно по разделам.

Метод	Путь	Назначение	Доступ
POST	/auth/login	Вход	публичный
POST	/auth/refresh	Обновить токены	публичный (refreshToken)
POST	/auth/logout	Выход	любой вошедший
GET	/auth/me	Текущий пользователь	любой вошедший
POST	/auth/change-password	Сменить свой пароль	любой вошедший
GET	/dashboard	Сводка за период	dashboard ≥ 1
GET	/rooms	Все номера с проживающими	rooms ≥ 1
GET	/rooms/available	Номера для заселения	registration ≥ 2
POST	/rooms	Добавить номер	admin
PUT	/rooms/{number}	Изменить номер	admin
POST	/rooms/{number}/ready	Номер готов (после уборки/ремонта)	rooms ≥ 2
PATCH	/rooms/{number}/status	Поставить на уборку/ремонт (на будущее)	rooms ≥ 2
POST	/rooms/{number}/extend	Продлить проживание	registration ≥ 2
POST	/rooms/{number}/checkout	Выселить (закрыть клиента)	registration ≥ 2
GET	/book	Все заселённые гости	book ≥ 1
POST	/book	Регистрация с заселением (1–10 гостей)	registration ≥ 2
PUT	/book/{id}	Изменить данные гостя	registration ≥ 2
GET	/guests/archive	Архив выехавших гостей	archive ≥ 1
GET	/guests/lookup	Найти гостя по документу (лупа)	registration ≥ 2
GET	/blacklist	Весь чёрный список	blacklist ≥ 1 или registration ≥ 2
GET	/blacklist/check	Проверить документы	registration ≥ 2
POST	/blacklist	Добавить в чёрный список	blacklist = 2
DELETE	/blacklist/{id}	Убрать из чёрного списка	blacklist = 2
POST	/fiscal/receipts	Получить номер чека	registration ≥ 2
GET	/expenses	Список расходов	expenses ≥ 1
GET	/expenses/recent	Последние расходы (дашборд)	dashboard ≥ 1
POST	/expenses	Добавить расход	expenses = 2
PUT	/expenses/{id}	Изменить расход	expenses = 2
DELETE	/expenses/{id}	Удалить расход	expenses = 2
GET	/staff	Сотрудники для выпадающих списков	любой вошедший
GET	/users	Все сотрудники	admin
GET	/roles/permissions	Матрица прав (только чтение)	admin
POST	/users	Добавить сотрудника	admin
PUT	/users/{id}	Изменить сотрудника	admin
PATCH	/users/{id}/active	Заблокировать / разблокировать	admin

Метод	Путь	Назначение	Доступ
POST	/users/{id}/reset-password	Сбросить пароль	admin
DELETE	/users/{id}	Удалить сотрудника	admin
GET	/settings/hotel	Данные отеля	любой вошедший
PUT	/settings/hotel	Сохранить данные отеля	admin
GET	/categories	Категории и тарифы	любой вошедший
POST	/categories	Добавить категорию	admin
PUT	/categories/{id}	Изменить категорию	admin
DELETE	/categories/{id}	Удалить категорию	admin
GET	/audit-log	Журнал действий	admin
GET	/audit-log/authors	Сотрудники в журнале (фильтр)	admin
GET	/comments	Все комментарии	любой вошедший
POST	/comments	Оставить комментарий (страница гостя /c/{room})	публичный
PATCH	/comments/{id}	Отметить прочитанным / новым	любой вошедший
POST	/comments/read-all	Отметить все прочитанными	любой вошедший
DELETE	/comments/{id}	Удалить комментарий	admin
GET	/qr	Номера с включённым QR	любой вошедший
GET	/qr/{room}	Включён ли QR (страница гостя)	публичный
PUT	/qr/{room}	Включить / выключить QR	admin

7.1. Авторизация

Вход по логину и паролю, JWT. Сейчас на фронте — моковый вход (любой активный логин + пароль от 4 символов).

POST /api/v1/auth/login — **Вход**

Доступ: публичный · **Тело:** LoginRequest · **Ответ 200:** LoginResponse

- Логин без учёта регистра, trim.
- Обновить lastLogin.
- remember = true — refresh-токен на 30 дней, иначе — на сессию (например, 12 часов).
- Защита от перебора: не больше 10 попыток в минуту с одного IP/логина.

HTTP	Код ошибки	Когда
401	auth.invalid	нет такого логина или пароль не подходит
403	auth.blocked	сотрудник заблокирован (active = false)

Пример запроса

```
{
  "login": "admin",
  "password": "••••••",
  "remember": true
}
```

Пример ответа

```
{
  "accessToken": "eyJhbGciOi…",
  "refreshToken": "c2f1…",
  "expiresIn": 900,
  "user": {
    "id": 1, "name": "Дилноза Каримова", "login": "admin",
    "role": "admin", "hotelName": "Navruz Hotel", "hotelCity": "Самарканд"
  }
}
```

Где на фронте: `core/auth/auth.service.ts → login()`

`POST /api/v1/auth/refresh` — **Обновить токены**

Доступ: публичный (refreshToken) · **Тело:** `RefreshRequest` · **Ответ 200:** `TokenPair`

- Ротация: старый refresh-токен становится недействительным.

HTTP	Код ошибки	Когда
401	`auth.unauthorized`	refresh-токен неверный, отозван или истёк

`POST /api/v1/auth/logout` — **Выход**

Доступ: любой вошедший · **Тело:** `RefreshRequest` · **Ответ 204:** без тела

- Отозвать refresh-токен.

Где на фронте: `кнопка «Выйти»`

`GET /api/v1/auth/me` — **Текущий пользователь**

Доступ: любой вошедший · **Ответ 200:** `CurrentUser`

- Фронт вызывает при старте, чтобы проверить сессию.

`POST /api/v1/auth/change-password` — **Сменить свой пароль**

Доступ: любой вошедший · **Тело:** `ChangePasswordRequest` · **Ответ 204:** без тела

- После успеха mustChangePassword = false.

HTTP	Код ошибки	Когда
422	`validation.failed`	старый пароль неверный или новый короче 6 символов

7.2. Дашборд

Одна сводка за выбранный период (неделя / месяц / произвольный).

`GET /api/v1/dashboard` — **Сводка за период**

Доступ: dashboard ≥ 1 · **Ответ 200:** `DashboardSummary`

Query-параметр	Тип	Обяз.	Описание
`from`	`IsoDate`	да	Начало периода включительно
`to`	`IsoDate`	да	Конец периода включительно

- occupancy — доля занятых номеро-ночей за период от продаваемых (номера не на ремонте), округлить до целого %.
- occupancyDelta — occupancy минус загрузка за предыдущий период такой же длины.
- revenue по дню — сумма amount выселений (CheckoutRequest.amount) с датой выезда в этот день.
- expenses по дню — сумма расходов с этой датой.
- daily — каждый день периода, дни без данных = 0.
- «Сегодня» — серверная дата в часовом поясе отеля (Asia/Tashkent).

HTTP	Код ошибки	Когда
422	validation.failed	from > to или период больше 366 дней

Где на фронте: core/data/dashboard.service.ts → summary(from, to)

7.3. Номера

Номерной фонд, карточка номера, продление, выселение, уборка.

GET /api/v1/rooms — **Все номера с проживающими**

Доступ: rooms ≥ 1 · **Ответ 200:** Room[]

- Сортировка по номеру (естественная: 101, 102, ..., 1001).
- guests — текущие гости из «Книги» в этом номере.
- Статус busy, если есть хотя бы один гость.

Пример ответа

```
[
  {
    "number": "301", "floor": 3, "category": "Семейный", "beds": 4,
    "status": "busy", "note": "", "price": null,
    "guests": [
      { "name": "Тошматов Алишер", "country": "UZ", "doc": "AA 1180342",
        "phone": "+998 91 300 70 70", "checkin": "2026-10-23T14:00",
        "checkout": "2026-10-29T12:00", "sum": 4800000, "pay": "part" }
    ]
  }
]
```

Где на фронте: core/data/rooms.service.ts → list()

GET /api/v1/rooms/available — **Номера для заселения**

Доступ: registration ≥ 2 · **Ответ 200:** Room[]

- Свободные (free) и занятые (busy), где guests.length < beds.
- Используется в выпадающем списке «Номер комнаты» формы регистрации.

Где на фронте: rooms.service.ts → checkInRooms()

POST /api/v1/rooms — **Добавить номер**

Доступ: admin · **Тело:** RoomDraft · **Ответ 201:** Room

- Новый номер — status free, note "", guests [].

HTTP	Код ошибки	Когда
409	rooms.exists	номер уже есть
422	validation.failed	неверный формат номера, нет категории

PUT /api/v1/rooms/{number} — **Изменить номер**

Доступ: admin · **Тело:** RoomDraft · **Ответ 200:** Room

- Можно переименовать (number в теле ≠ в пути): обновить ссылки у гостей книги, QR-кодов, комментариев.
- Статус и гости не меняются.

HTTP	Код ошибки	Когда
404	common.notFound	
409	rooms.exists	новый номер уже занят
409	rooms.hasGuests	beds меньше числа проживающих

`POST` `/api/v1/rooms/{number}/ready` — Номер готов (после уборки/ремонта)

Доступ: rooms ≥ 2 · **Ответ 200:** `Room`

- clean/repair → free, note = "".
- Журнал: ready { room }.

HTTP	Код ошибки	Когда
409	`rooms.invalidStatus`	номер не в статусе clean или repair

Где на фронте: `кнопки «Отметить как готов», «Завершить ремонт»`

`PATCH` `/api/v1/rooms/{number}/status` — Поставить на уборку/ремонт (на будущее)

Доступ: rooms ≥ 2 · **Тело:** `RoomStatusUpdate` · **Ответ 200:** `Room`

- В интерфейсе пока нет — можно сделать позже.
- Только free → clean | repair.

HTTP	Код ошибки	Когда
409	`rooms.hasGuests`	в номере гости

`POST` `/api/v1/rooms/{number}/extend` — Продлить проживание

Доступ: registration ≥ 2 · **Тело:** `ExtendRequest` · **Ответ 200:** `Room`

- Новая дата выезда — у всех гостей номера (и в их регистрациях в книге).
- Журнал: bookingEdit { guest: первый гость, change: «30 → 31.10» }.

HTTP	Код ошибки	Когда
409	`rooms.invalidStatus`	в номере нет гостей
422	`validation.dates`	checkout ≤ заезда

`POST` `/api/v1/rooms/{number}/checkout` — Выселить (закрыть клиента)

Доступ: registration ≥ 2 · **Тело:** `CheckoutRequest` · **Ответ 200:** `Room`

- Всё в одной транзакции:
- 1) выезжающие уходят из книги в архив (ArchivedGuest): checkin — дата заезда, checkout — сегодня, sum — amount у первого выезжающего, у остальных 0;
- 2) blacklistReason не null — каждый выезжающий в чёрный список (уже внесённые пропустить), addedBy — текущий сотрудник;
- 3) уехали все — status clean, note "", guests []; остались гости — номер остаётся busy;
- 4) сохранить запись выселения: room, docs, amount, receipt, кто и когда — нужна для выручки на дашборде;
- 5) журнал: checkout { guest, room } на каждого.
- Ответ — номер после выселения.

HTTP	Код ошибки	Когда
409	`rooms.invalidStatus`	в номере нет гостей
422	`validation.failed`	docs пустой или есть документ, которого нет в номере

Пример запроса

```
{
  "docs": ["AA 1180977", "AD 5520114"],
  "amount": 4800000,
  "receipt": "001041",
  "blacklistReason": null
}
```

Где на фронте: `rooms.ts → confirmCheckout() → book.service.ts → checkout()`

7.4. Книга гостей

Все заселённые гости. Регистрация сразу заселяет в номер. Выезд — только через выселение номера.

GET /api/v1/book — **Все заселённые гости**

Доступ: book ≥ 1 · **Ответ 200:** `BookEntry[]`

- Новые сверху (по id убыв.).

Где на фронте: `core/data/book.service.ts → list()`

POST /api/v1/book — **Регистрация с заселением (1–10 гостей)**

Доступ: registration ≥ 2 · **Тело:** `BookCreateRequest` · **Ответ 201:** `BookEntry[]`

- Атомарно: либо все гости заселены, либо ни один.
- Гости могут быть в разных номерах (у каждого свой room) — обычно в одном.
- Документ нормализовать для сравнений (без пробелов, верхний регистр), хранить как прислали.
- Номер → busy, гость появляется в Room.guests: sum = amount, pay = wait если payment = unpaid, иначе paid.
- Сохранить/обновить профиль гостя по документу — для GET /guests/lookup.
- Журнал: checkin { guest: «Фамилия Имя», room } на каждого.
- fiscalReceipt и sendEmehmon — только сохранить (сейчас всегда false).

HTTP	Код ошибки	Когда
422	`validation.failed`	нет doc/lastName/firstName/room или amount < 0
422	`validation.dates`	checkout ≤ checkin
422	`guests.duplicateDoc`	один документ дважды в запросе
404	`common.notFound`	номер не существует
409	`rooms.unavailable`	номер на уборке/ремонте
409	`rooms.full`	мест не хватает: уже живущие + новые > beds
409	`guests.alreadyIn`	гость с этим документом уже проживает
409	`blacklist.found`	документ в ЧС и confirmBlacklisted ≠ true

Пример запроса

```
{
  "registrations": [{
    "docType": "bio_passport", "doc": "AB 1234567",
    "issuedAt": "2019-05-14", "expiresAt": "2029-05-13",
    "issuedBy": "ОВД Мирзо-Улугбекского района", "lastName": "Иванов",
    "firstName": "Пётр", "middleName": "", "birthDate": "1990-02-01",
    "sex": "male", "country": "UZ", "nationality": "узбек",
    "phone": "+998 90 123 45 67", "cityFrom": "Ташкент", "address": "…",
    "foreign": false, "visaType": null, "entryDate": null, "entryPoint": "",
    "room": "101", "checkin": "2026-10-26T14:00", "checkout": "2026-10-30T12:00",
    "purpose": "tourism", "amount": 1900000, "payment": "cash",
    "fiscalReceipt": false, "sendEmehmon": false
  }],
  "confirmBlacklisted": false
}
```

Пример ответа

```
[ { "id": 14, "reg": { …то же, что прислали… } } ]
```

Где на фронте: `registration-tabs.ts → save() → book.service.ts → register()`

PUT /api/v1/book/{id} — **Изменить данные гостя**

Доступ: registration ≥ 2 · **Тело:** `GuestRegistration` · **Ответ 200:** `BookEntry`

- Если room изменился — гость переезжает: убрать из старого номера (пустой → clean), добавить в новый.
- Обновить карточку номера (RoomGuest) и профиль гостя.

HTTP	Код ошибки	Когда
404	`common.notFound`	
422	`validation.dates`	
409	`rooms.full`	перевод в номер без мест
409	`rooms.unavailable`	

Где на фронте: `book.ts → saveEdit()`

7.5. Гости (архив)

Выехавшие гости и поиск по документу для автозаполнения формы.

GET /api/v1/guests/archive — **Архив выехавших гостей**

Доступ: archive ≥ 1 · **Ответ 200:** `ArchivedGuest[]`

- Сортировка по дате выезда, новые сверху.

Где на фронте: `core/data/guests.service.ts → archive()`

GET /api/v1/guests/lookup — **Найти гостя по документу (лупа)**

Доступ: registration ≥ 2 · **Ответ 200:** `GuestProfile`

Query-параметр	Тип	Обяз.	Описание
`doc`	`string`	да	Серия и номер, пробелы и регистр не важны

- Искать по нормализованному документу среди всех известных профилей (архив и книга), брать самый свежий.

HTTP	Код ошибки	Когда
404	`guests.notFound`	гостя с таким документом не было

Где на фронте: `registration-form.ts → lookup(), blacklist-dialog.ts → lookup()`

7.6. Чёрный список

Гости, которым отказано в заселении. Сверка по документу при регистрации.

GET /api/v1/blacklist — **Весь чёрный список**

Доступ: blacklist ≥ 1 или registration ≥ 2 · **Ответ 200:** `BlacklistEntry[]`

- Новые сверху.
- Ресепшн нужен список (или check) для предупреждения при заселении.

Где на фронте: `core/data/blacklist.service.ts → list()`

`GET` **`/api/v1/blacklist/check`** — **Проверить документы**

Доступ: registration ≥ 2 · **Ответ 200:** `BlacklistEntry[]`

Query-параметр	Тип	Обяз.	Описание
`doc`	`string[]`	да	Один или несколько: ?doc=AB1234567&doc=AC7654321

- Возвращает только совпадения (пустой массив — всё чисто).
- Фронт вызывает перед заселением и после «лупы», чтобы показать модалку-предупреждение.

Где на фронте: `blacklist.service.ts → find() (сейчас синхронно по загруженному списку)`

`POST` **`/api/v1/blacklist`** — **Добавить в чёрный список**

Доступ: blacklist = 2 · **Тело:** `BlacklistDraft` · **Ответ 201:** `BlacklistEntry`

- addedAt — сейчас, addedBy — ФИО текущего сотрудника.
- Добавляют из страницы ЧС, «Книги», «Гостей» и при выселении (через checkout).

HTTP	Код ошибки	Когда
409	`blacklist.exists`	документ уже в списке
422	`validation.failed`	нет doc/фамилии/имени/причины

`DELETE` **`/api/v1/blacklist/{id}`** — **Убрать из чёрного списка**

Доступ: blacklist = 2 · **Ответ 204:** без тела

HTTP	Код ошибки	Когда
404	`common.notFound`	

7.7. Фискальный чек

Номер чека при выселении. Сам Word-документ формирует фронт по шаблону.

`POST` **`/api/v1/fiscal/receipts`** — **Получить номер чека**

Доступ: registration ≥ 2 · **Тело:** `ReceiptRequest` · **Ответ 201:** `Receipt`

- Номер — сквозной счётчик отеля, 6 цифр с ведущими нулями.
- Сохранить чек; при выселении номер чека придёт в CheckoutRequest.receipt.
- Word-файл (бланк «Счёт» гостиницы) формирует фронт: public/templates/fiscal-receipt.docx. Бэкенду PDF/Word делать не нужно.
- Подключение онлайн-ККМ / ОФД — отдельная задача, пока не нужно.

HTTP	Код ошибки	Когда
422	`validation.failed`	amount ≤ 0, нет номера или гостей

Пример ответа

```
{ "number": "001041", "room": "301", "amount": 4800000, "createdAt": "2026-10-26T11:40" }
```

Где на фронте: `rooms.ts → generateReceipt() → book.service.ts → fiscalReceipt()`

7.8. Расходы

Учёт расходов отеля.

`GET` `/api/v1/expenses` — **Список расходов**

Доступ: expenses ≥ 1 · **Ответ 200:** `Expense[]`

Query-параметр	Тип	Обяз.	Описание
`from`	`IsoDate`	нет	
`to`	`IsoDate`	нет	
`who`	`string`	нет	ФИО сотрудника

- Новые сверху (по date, затем id).

Где на фронте: `core/data/expenses.service.ts → list()`

`GET` `/api/v1/expenses/recent` — **Последние расходы (дашборд)**

Доступ: dashboard ≥ 1 · **Ответ 200:** `Expense[]`

Query-параметр	Тип	Обяз.	Описание
`limit`	`integer`	нет	По умолчанию 5

Где на фронте: `expenses.service.ts → recent()`

`POST` `/api/v1/expenses` — **Добавить расход**

Доступ: expenses = 2 · **Тело:** `ExpenseDraft` · **Ответ 201:** `Expense`

- Журнал: expense { reason, sum: «3 200 000» }.

HTTP	Код ошибки	Когда
422	`validation.failed`	нет reason, sum ≤ 0, нет даты

`PUT` `/api/v1/expenses/{id}` — **Изменить расход**

Доступ: expenses = 2 · **Тело:** `ExpenseDraft` · **Ответ 200:** `Expense`

HTTP	Код ошибки	Когда
404	`common.notFound`	

`DELETE` `/api/v1/expenses/{id}` — **Удалить расход**

Доступ: expenses = 2 · **Ответ 204:** без тела

HTTP	Код ошибки	Когда
404	`common.notFound`	

`GET` `/api/v1/staff` — **Сотрудники для выпадающих списков**

Доступ: любой вошедший · **Ответ 200:** `StaffShort[]`

- Только активные. Нужен в «Расходах» (кто израсходовал) и фильтре журнала. Сейчас на фронте — MOCK_STAFF_NAMES.

7.9. Сотрудники и роли

Только owner/admin. Матрица прав — только чтение.

`GET` `/api/v1/users` — **Все сотрудники**

Доступ: admin · **Ответ 200:** `StaffUser[]`

- online — был запрос с токеном этого сотрудника за последние 5 минут.

Где на фронте: `core/data/users.service.ts`

`GET` `/api/v1/roles/permissions` — **Матрица прав (только чтение)**

Доступ: admin · **Ответ 200:** `PermissionRow[]`

- Те же значения, по которым бэкенд проверяет доступ (таблица «Роли и доступ»).

`POST` `/api/v1/users` — **Добавить сотрудника**

Доступ: admin · **Тело:** `StaffUserDraft` · **Ответ 201:** `StaffUser`

- Пароль хранить только хэшем (bcrypt/argon2).
- mustChangePassword = true.

HTTP	Код ошибки	Когда
409	`users.loginTaken`	
422	`validation.failed`	логин не по шаблону, пароль < 6

`PUT` `/api/v1/users/{id}` — **Изменить сотрудника**

Доступ: admin · **Тело:** `StaffUserDraft` · **Ответ 200:** `StaffUser`

- password — только если прислан.

HTTP	Код ошибки	Когда
404	`common.notFound`	
409	`users.loginTaken`	
403	`users.self`	снять active у себя

`PATCH` `/api/v1/users/{id}/active` — **Заблокировать / разблокировать**

Доступ: admin · **Тело:** `ActiveUpdate` · **Ответ 200:** `StaffUser`

- При блокировке отозвать все токены сотрудника.
- Журнал: block { user } при active = false.

HTTP	Код ошибки	Когда
403	`users.self`	свою учётную запись

`POST` `/api/v1/users/{id}/reset-password` — **Сбросить пароль**

Доступ: admin · **Ответ 204:** без тела

- Сгенерировать временный пароль и отправить SMS на phone (фронт пишет «Временный пароль отправлен по SMS»).
- mustChangePassword = true. Если SMS пока нет — вернуть 200 { "temporaryPassword": "..." } и договориться с фронтом.

HTTP	Код ошибки	Когда
404	`common.notFound`	

`DELETE` **/api/v1/users/{id}** — **Удалить сотрудника**

Доступ: admin · **Ответ 204:** без тела

- Лучше мягкое удаление: в журнале и расходах остаётся ФИО.

HTTP	Код ошибки	Когда
403	`users.self`	себя
404	`common.notFound`	

7.10. Настройки

Данные отеля, категории и тарифы, журнал действий.

`GET` **/api/v1/settings/hotel** — **Данные отеля**

Доступ: любой вошедший · **Ответ 200:** `HotelSettings`

- Нужны всем: название в шапке, время заезда/выезда.

`PUT` **/api/v1/settings/hotel** — **Сохранить данные отеля**

Доступ: admin · **Тело:** `HotelSettings` · **Ответ 200:** `HotelSettings`

HTTP	Код ошибки	Когда
422	`settings.innInvalid`	ИНН не пустой и не 9 цифр

`GET` **/api/v1/categories** — **Категории и тарифы**

Доступ: любой вошедший · **Ответ 200:** `RoomCategory[]`

- rooms — сколько номеров в категории.

`POST` **/api/v1/categories** — **Добавить категорию**

Доступ: admin · **Тело:** `CategoryDraft` · **Ответ 201:** `RoomCategory`

HTTP	Код ошибки	Когда
409	`settings.categoryExists`	название без учёта регистра

`PUT` **/api/v1/categories/{id}** — **Изменить категорию**

Доступ: admin · **Тело:** `CategoryDraft` · **Ответ 200:** `RoomCategory`

- Переименование — обновить category у номеров.
- Цена изменилась — журнал tariffEdit.

HTTP	Код ошибки	Когда
404	`common.notFound`	
409	`settings.categoryExists`	

`DELETE` **/api/v1/categories/{id}** — **Удалить категорию**

Доступ: admin · **Ответ 204:** без тела

HTTP	Код ошибки	Когда
409	`settings.categoryInUse`	есть номера этой категории

`GET` **/api/v1/audit-log** — **Журнал действий**

Доступ: admin · **Ответ 200:** `ActionLogEntry[]`

Query-параметр	Тип	Обяз.	Описание
`from`	`IsoDate`	нет	
`to`	`IsoDate`	нет	
`who`	`string`	нет	ФИО

- Новые сверху. Записи создаёт сам бэкенд (таблица событий).

`GET` **/api/v1/audit-log/authors** — **Сотрудники в журнале (фильтр)**

Доступ: admin · **Ответ 200:** `string[]`

7.11. Комментарии и QR-коды

Уже работает в server/hotel-api.mjs (без авторизации) — перенести в бэкенд. Отзывы гостей по QR-коду в номере.

`GET` **/api/v1/comments** — **Все комментарии**

Доступ: любой вошедший · **Ответ 200:** `GuestComment[]`

- Новые сверху. Фронт опрашивает каждые 10 секунд (бейдж «новых» в меню).

`POST` **/api/v1/comments** — **Оставить комментарий (страница гостя /c/{room})**

Доступ: публичный · **Тело:** `CommentDraft` · **Ответ 201:** `GuestComment`

- Без авторизации — ограничить частоту (например, 5 в минуту с IP).
- status = new.

HTTP	Код ошибки	Когда
403	`comments.qrDisabled`	QR для номера выключен
422	`validation.failed`	rating не 1–5, пустой текст

`PATCH` **/api/v1/comments/{id}** — **Отметить прочитанным / новым**

Доступ: любой вошедший · **Тело:** `CommentStatusUpdate` · **Ответ 200:** `GuestComment`

`POST` **`/api/v1/comments/read-all`**	**Отметить все прочитанными**

Доступ: любой вошедший · **Ответ 204:** без тела

`DELETE` **`/api/v1/comments/{id}`**	**Удалить комментарий**

Доступ: admin · **Ответ 204:** без тела

`GET` **`/api/v1/qr`**	**Номера с включённым QR**

Доступ: любой вошедший · **Ответ 200:** `string[]`

`GET` **`/api/v1/qr/{room}`**	**Включён ли QR (страница гостя)**

Доступ: публичный · **Ответ 200:** `QrState`

`PUT` **`/api/v1/qr/{room}`**	**Включить / выключить QR**

Доступ: admin · **Тело:** `QrUpdate` · **Ответ 200:** `QrState`

- from — прежний номер при переименовании комнаты (перенести состояние).

8. Модели данных

Типы: `string`, `integer`, `boolean`, `IsoDate`, `IsoDateTime` (раздел 3), имя модели или перечисления, `X[]` — массив, `X|null` — может быть null, `map:string` — объект «ключ → строка». «Обяз.: да» — поле всегда есть в ответе и обязательно в запросе.

8.1. Модели

CurrentUser — Текущий пользователь (после входа).

Поле	Тип	Обяз.	Описание
`id`	`integer`	да	
`name`	`string`	да	ФИО сотрудника
`login`	`string`	да	
`role`	`RoleCode`	да	
`hotelName`	`string`	да	Название отеля (из настроек)
`hotelCity`	`string`	да	Город отеля
`mustChangePassword`	`boolean`	нет	true — после сброса пароля, фронт попросит сменить (новое поле)

LoginRequest — Вход по логину и паролю.

Поле	Тип	Обяз.	Описание
`login`	`string`	да	Без учёта регистра, пробелы по краям обрезаются
`password`	`string`	да	Минимум 4 символа
`remember`	`boolean`	да	true — долгая сессия (refresh 30 дней), false — до закрытия браузера

LoginResponse — Ответ на вход.

Поле	Тип	Обяз.	Описание
`accessToken`	`string`	да	
`refreshToken`	`string`	да	
`expiresIn`	`integer`	да	
`user`	`CurrentUser`	да	

TokenPair — Токены доступа.

Поле	Тип	Обяз.	Описание
`accessToken`	`string`	да	JWT, живёт ~15 минут
`refreshToken`	`string`	да	Для POST /auth/refresh
`expiresIn`	`integer`	да	Срок жизни accessToken в секундах

RefreshRequest — Токен обновления.

Поле	Тип	Обяз.	Описание
`refreshToken`	`string`	да	

ChangePasswordRequest — Смена пароля.

Поле	Тип	Обяз.	Описание
oldPassword	string	да	
newPassword	string	да	≥ 6

Room — Номер (комната).

Поле	Тип	Обяз.	Описание
number	string	да	Идентификатор в URL: «206», «12A»
floor	integer	да	
category	string	да	Название категории
beds	integer	да	Количество мест
status	RoomStatus	да	
note	string	да	Подсказка на карточке: «Уборка: Гулнора», «Мастер вызван». Может быть пустой
price	integer|null	нет	Свой тариф номера; null — тариф категории (новое поле)
guests	RoomGuest[]	да	Все проживающие; пусто — номер не занят

RoomGuest — Проживающий в номере (краткие данные для карточки номера).

Поле	Тип	Обяз.	Описание
name	string	да	«Фамилия Имя»
country	string	да	ISO 3166-1 alpha-2
doc	string	да	Серия и номер документа
phone	string	да	Может быть пустой строкой
checkin	IsoDateTime	да	
checkout	IsoDateTime	да	
sum	integer	да	Сумма по регистрации этого гостя, сум
pay	PaymentStatus	да	unpaid → wait, иначе paid

RoomDraft — Создание/изменение номера.

Поле	Тип	Обяз.	Описание
number	string	да	Шаблон ^\d{1,4}[A-Za-zA-Яа-я]?$
floor	integer	да	1–5
category	string	да	Существующая категория
beds	integer	да	1–4
price	integer|null	да	null — брать тариф категории

RoomStatusUpdate — Новый статус номера.

Поле	Тип	Обяз.	Описание
status	RoomStatus	да	clean | repair
note	string	нет	Например «Не работает кондиционер»

ExtendRequest — Продление проживания.

Поле	Тип	Обяз.	Описание
checkout	IsoDateTime	да	Новая дата и время выезда для всех гостей номера

CheckoutRequest — Выселение (закрытие клиента).

Поле	Тип	Обяз.	Описание
docs	string[]	да	Документы выезжающих гостей (не пусто). Все гости — номер уходит на уборку
amount	integer	да	Итоговая сумма, сум (≥ 0)
receipt	string|null	да	Номер фискального чека (из POST /fiscal/receipts) или null
blacklistReason	string|null	да	Не null — добавить всех выезжающих в чёрный список с этой причиной

GuestProfile — Данные гостя по документу (автозаполнение формы по «лупе»).

Поле	Тип	Обяз.	Описание
docType	DocType	да	
doc	string	да	Серия и номер, например «AB 1234567»
issuedAt	IsoDate|null	да	Дата выдачи
expiresAt	IsoDate|null	да	Действителен до
issuedBy	string	да	Кем выдан
lastName	string	да	
firstName	string	да	
middleName	string	да	Может быть пустой
birthDate	IsoDate|null	да	
sex	Sex	да	
country	string	да	Гражданство, ISO 3166-1 alpha-2
nationality	string	да	Национальность текстом
phone	string	да	
cityFrom	string	да	Откуда прибыл
address	string	да	Адрес постоянного проживания

GuestRegistration — Регистрация гостя = все поля GuestProfile + поля ниже.

Поле	Тип	Обяз.	Описание
…	GuestProfile	да	Все 15 полей GuestProfile
foreign	boolean	да	Иностранец (не гражданин РУз)
visaType	VisaType|null	да	Только для иностранцев, иначе null
entryDate	IsoDate|null	да	Дата въезда в РУз (иностранцы)
entryPoint	string	да	Пункт въезда (иностранцы), иначе пусто
room	string	да	Номер комнаты
checkin	IsoDateTime	да	Заезд
checkout	IsoDateTime	да	Выезд, строго позже заезда
purpose	VisitPurpose	да	
amount	integer	да	Договорная сумма за весь период, сум (≥ 0)
payment	PaymentMethod	да	
fiscalReceipt	boolean	да	Сейчас всегда false (чек — при выселении). Хранить, не обрабатывать
sendEmehmon	boolean	да	Сейчас всегда false. Хранить, интеграцию с E-mehmon не делать

BookEntry — Запись «Книги гостей» — заселённый гость.

Поле	Тип	Обяз.	Описание
id	integer	да	
reg	GuestRegistration	да	Полные данные регистрации

BookCreateRequest — Регистрация с заселением одного или нескольких гостей (табы в модалке).

Поле	Тип	Обяз.	Описание
registrations	GuestRegistration[]	да	От 1 до 10 гостей; сохраняются атомарно
confirmBlacklisted	boolean	нет	true — сотрудник видел предупреждение о ЧС и всё равно заселяет

ArchivedGuest — Гость в архиве (после выезда).

Поле	Тип	Обяз.	Описание
id	integer	да	
lastName	string	да	
firstName	string	да	
middleName	string	да	
doc	string	да	
country	string	да	
room	string	да	
checkin	IsoDate	да	
checkout	IsoDate	да	Фактическая дата выезда
sum	integer	да	Итог выселения у первого выезжающего, у остальных 0
birthDate	IsoDate	да	Пустая строка, если неизвестна
sex	Sex	да	
phone	string	да	

BlacklistEntry — Гость в чёрном списке. Сверка — по документу.

Поле	Тип	Обяз.	Описание
id	integer	да	
doc	string	да	
lastName	string	да	
firstName	string	да	
middleName	string	да	
country	string	да	
reason	string	да	Причина, обязательна
addedAt	IsoDateTime	да	Ставит сервер
addedBy	string	да	ФИО сотрудника, ставит сервер

BlacklistDraft — Добавление в чёрный список.

Поле	Тип	Обяз.	Описание
doc	string	да	
lastName	string	да	
firstName	string	да	
middleName	string	да	
country	string	да	
reason	string	да	Не пустая (после trim)

ReceiptRequest — Запрос номера фискального чека.

Поле	Тип	Обяз.	Описание
room	string	да	
amount	integer	да	> 0
docs	string[]	да	Документы выезжающих гостей

Receipt — Фискальный чек (номер счёта).

Поле	Тип	Обяз.	Описание
number	string	да	6 цифр с ведущими нулями, сквозная нумерация: «001041»
room	string	да	
amount	integer	да	
createdAt	IsoDateTime	да	

Expense — Расход.

Поле	Тип	Обяз.	Описание
id	integer	да	
reason	string	да	На что потрачено
sum	integer	да	> 0, сум
date	IsoDate	да	
who	string	да	ФИО сотрудника, кто израсходовал

ExpenseDraft — Создание/изменение расхода.

Поле	Тип	Обяз.	Описание
reason	string	да	
sum	integer	да	
date	IsoDate	да	
who	string	да	ФИО из GET /staff

StaffUser — Сотрудник (пользователь системы).

Поле	Тип	Обяз.	Описание
id	integer	да	
name	string	да	
login	string	да	
role	RoleCode	да	
phone	string	да	
active	boolean	да	false — заблокирован, войти не может
lastLogin	IsoDateTime|null	да	null — ещё не входил
online	boolean	да	Был активен за последние 5 минут

StaffUserDraft — Создание/изменение сотрудника.

Поле	Тип	Обяз.	Описание
name	string	да	
login	string	да	Шаблон ^[a-z0-9._-]{3,32}$, уникальный
role	RoleCode	да	
phone	string	да	
active	boolean	да	
password	string	нет	Обязателен при создании (≥ 6), при изменении — только если меняют

StaffShort — Сотрудник для выпадающих списков.

Поле	Тип	Обяз.	Описание
id	integer	да	
name	string	да	
role	RoleCode	да	

ActiveUpdate — Блокировка.

Поле	Тип	Обяз.	Описание
active	boolean	да	

PermissionRow — Строка матрицы прав.

Поле	Тип	Обяз.	Описание
section	string	да	dashboard | registration | rooms | book | blacklist | archive | expenses | admin
access	RoleAccess	да	

RoleAccess — Уровень доступа ролей к разделу: 2 — полный, 1 — просмотр, 0 — нет.

Поле	Тип	Обяз.	Описание
owner	integer	да	0 | 1 | 2
admin	integer	да	
reception	integer	да	
housekeeper	integer	да	
accountant	integer	да	

HotelSettings — Настройки отеля.

Поле	Тип	Обяз.	Описание
name	string	да	Название
legalName	string	да	Юр. лицо
inn	string	да	ИНН: 9 цифр или пусто
phone	string	да	
address	string	да	
checkinTime	string	да	«HH:mm», расчётное время заезда
checkoutTime	string	да	«HH:mm», расчётное время выезда
defaultLang	Lang	да	
currency	string	да	Всегда «UZS»

RoomCategory — Категория номеров и тариф.

Поле	Тип	Обяз.	Описание
id	integer	да	
name	string	да	Уникальное название: «Стандарт», «Делюкс»…
rooms	integer	да	Сколько номеров этой категории (вычисляется сервером)
beds	string	да	Текстом: «1–2», «4»
price	integer	да	Базовый тариф за ночь, сум

CategoryDraft — Создание/изменение категории.

Поле	Тип	Обяз.	Описание
name	string	да	
beds	string	да	
price	integer	да	> 0

ActionLogEntry — Событие журнала действий.

Поле	Тип	Обяз.	Описание
id	integer	да	
at	IsoDateTime	да	
who	string	да	ФИО сотрудника
action	LogAction	да	
params	map:string	да	Параметры для текста (см. таблицу событий)

DashboardSummary — Сводка дашборда за период.

Поле	Тип	Обяз.	Описание
occupancy	integer	да	Загрузка за период, %
occupancyDelta	integer	да	Изменение к предыдущему такому же периоду, п.п. (может быть < 0)
arrivalsToday	integer	да	Заезды сегодня
arrivalsWaitingPayment	integer	да	Из них с payment = unpaid
departuresToday	integer	да	Выезды сегодня (checkout сегодня)
lateCheckouts	integer	да	Выезды сегодня позже checkoutTime отеля
daily	DailyFinance[]	да	По каждому дню периода, без пропусков
fund	RoomFund	да	
totalRooms	integer	да	
categories	CategoryOccupancy[]	да	
arrivals	Arrival[]	да	Заезды сегодня

DailyFinance — Финансы за день.

Поле	Тип	Обяз.	Описание
date	IsoDate	да	
revenue	integer	да	Выручка, сум
expenses	integer	да	Расходы, сум

RoomFund — Номерной фонд сейчас: количество номеров по статусам.

Поле	Тип	Обяз.	Описание
free	integer	да	
busy	integer	да	
clean	integer	да	
repair	integer	да	

CategoryOccupancy — Загрузка категории за период.

Поле	Тип	Обяз.	Описание
name	string	да	
rooms	integer	да	
occupancy	integer	да	%, 0–100

Arrival — Заезд сегодня (таблица на дашборде).

Поле	Тип	Обяз.	Описание
guest	string	да	
room	string	да	
category	string	да	
from	IsoDate	да	
to	IsoDate	да	
sum	integer	да	
pay	PaymentStatus	да	

GuestComment — Комментарий гостя по QR-коду.

Поле	Тип	Обяз.	Описание
id	integer	да	
at	IsoDateTime	да	
room	string	да	
guest	string	да	Имя; пусто — аноним
rating	integer	да	1–5
text	string	да	≤ 500 символов
status	CommentStatus	да	

CommentDraft — Новый комментарий (публично, со страницы гостя).

Поле	Тип	Обяз.	Описание
room	string	да	
rating	integer	да	1–5
text	string	да	Не пустой, ≤ 500
guest	string	нет	≤ 60

CommentStatusUpdate — Статус комментария.

Поле	Тип	Обяз.	Описание
status	CommentStatus	да	

QrState — QR-код номера.

Поле	Тип	Обяз.	Описание
room	string	да	
enabled	boolean	да	

QrUpdate — Включить/выключить QR-код.

Поле	Тип	Обяз.	Описание
`enabled`	`boolean`	да	
`from`	`string`	нет	Прежний номер, если номер комнаты переименовали

Error — Ошибка. Фронт показывает текст по коду (errors.<code> в словарях).

Поле	Тип	Обяз.	Описание
`error.code`	`string`	да	Код из таблицы кодов, например «rooms.exists»
`error.message`	`string`	да	Текст для логов/разработчика (пользователю не показывается)
`error.params`	`map:string`	нет	Подстановки в текст: { "number": "206" }
`error.fields`	`map:string`	нет	Для 422: поле → код ошибки поля

8.2. Перечисления

Тип	Значения	Смысл
`RoleCode`	`owner | admin | reception | housekeeper | accountant`	Роль сотрудника: владелец, администратор, ресепшн, горничная, бухгалтер.
`RoomStatus`	`free | busy | clean | repair`	Статус номера: свободен, занят, уборка, ремонт.
`PaymentStatus`	`paid | part | wait`	Статус оплаты проживающего: оплачено, предоплата, ожидает оплаты.
`PaymentMethod`	`cash | uzcard | visa | transfer | unpaid`	Способ оплаты: наличные, Uzcard/Humo, Visa/Mastercard, перечисление, не оплачен.
`DocType`	`bio_passport | id_card | foreign_passport`	Тип документа: биометрический паспорт РУз, ID-карта РУз, заграничный паспорт.
`Sex`	`male | female`	Пол.
`VisitPurpose`	`tourism | business | private`	Цель визита.
`VisaType`	`visa_free | tourist | business`	Тип визы (только для иностранцев).
`Lang`	`ru | uz-Latn | uz-Cyrl`	Язык интерфейса.
`CommentStatus`	`new | read`	Статус комментария гостя.
`LogAction`	`checkin | checkout | ready | bookingEdit | tariffEdit | expense | block`	Тип события в журнале действий.

9. Журнал действий

Записи создаёт бэкенд. Фронт собирает текст события по action и params (переводы уже есть), поэтому имена параметров менять нельзя. who — ФИО сотрудника, at — время события.

action	params	Когда писать
`checkin`	`guest, room`	Каждый заселённый гость (POST /book)
`checkout`	`guest, room`	Каждый выселенный гость (POST /rooms/{n}/checkout)
`ready`	`room`	Номер отмечен готовым (POST /rooms/{n}/ready)
`bookingEdit`	`guest, change`	Продление: change = «30 → 31.10» (POST /rooms/{n}/extend)
`tariffEdit`	`category, change`	Изменилась цена категории: change = «600 000 → 640 000»
`expense`	`reason, sum`	Добавлен расход; sum — строкой с пробелами «3 200 000»
`block`	`user`	Сотрудник заблокирован (PATCH /users/{id}/active, active=false)

10. Подключение фронта (для справки)

Что фронт поменяет, когда API будет готов, — чтобы бэкенд знал, какие места зависят от его ответов.

Файл фронта	Методы → эндпоинты
`core/auth/auth.service.ts`	login, logout; добавить interceptor для Bearer и refresh
`core/data/rooms.service.ts`	list, checkInRooms, add, update, markReady, extend
`core/data/book.service.ts`	list, register, update, checkout, fiscalReceipt
`core/data/guests.service.ts`	archive, lookupByDoc
`core/data/blacklist.service.ts`	list, add, remove, find → GET /blacklist/check
`core/data/expenses.service.ts`	list, recent, create, update, remove
`core/data/users.service.ts`	list, permissions, create, update, setActive, resetPassword, remove
`core/data/settings.service.ts`	getHotel, saveHotel, categories CRUD, log, logAuthors
`core/data/dashboard.service.ts`	summary(from, to)
`core/data/comments.service.ts`, `room-qr.service.ts`	уже на HttpClient — поменять базовый путь на /api/v1

Новое относительно текущих моков (фронт добавит при подключении):

- HTTP-interceptor: заголовок Bearer, повтор запроса после POST /auth/refresh, выход при 401.
- Проверка ЧС через `GET /blacklist/check` вместо загруженного списка; повторная отправка `POST /book` с `confirmBlacklisted: true` после предупреждения.
- Список сотрудников из `GET /staff` вместо MOCK_STAFF_NAMES; дата «сегодня» — с сервера.
- Переводы для новых кодов ошибок (раздел 5.1, «новый»), смена пароля при mustChangePassword.
- Комментарии и QR: базовый путь `/api` → `/api/v1` (сейчас их отдаёт server/hotel-api.mjs).

11. Рекомендуемая схема БД

Не обязательна — ориентир для старта (PostgreSQL). Названия таблиц и колонок можно выбрать свои; важно только API.

Таблица	Колонки
`users`	id, name, login (unique), password_hash, role, phone, active, must_change_password, last_login_at, last_seen_at, deleted_at
`refresh_tokens`	id, user_id, token_hash, expires_at, revoked_at
`hotel_settings`	одна строка: name, legal_name, inn, phone, address, checkin_time, checkout_time, default_lang, currency
`room_categories`	id, name (unique ci), beds_text, price
`rooms`	number (pk), floor, category_id, beds, status, note, price (nullable), qr_enabled
`guest_profiles`	doc_norm (pk), doc, все поля GuestProfile, updated_at — для «лупы»
`stays`	= BookEntry: id, room_number, doc_norm, registration (jsonb или колонки), created_by, created_at; unique(doc_norm) среди активных
`checkouts`	id, room_number, amount, receipt_number, blacklisted, created_by, created_at
`archived_guests`	= ArchivedGuest + checkout_id
`receipts`	number (pk, sequence), room_number, amount, docs, created_by, created_at
`blacklist`	id, doc, doc_norm (unique), last_name, first_name, middle_name, country, reason, added_by, added_at
`expenses`	id, reason, sum, date, who
`audit_log`	id, at, user_id, who, action, params (jsonb)

Таблица	Колонки
`comments`	id, at, room_number, guest, rating, text, status

12. Чек-лист приёмки

- openapi.yaml проходит валидацию; ответы совпадают со схемами (имена полей, типы, перечисления).
- Даты/время — в формате раздела 3, в часовом поясе отеля.
- Ошибки — в формате раздела 5, с кодами из таблицы 5.1.
- Права ролей проверяются на сервере по матрице 4.2 (GET разрешён при уровне 1, изменения — при 2).
- POST /book атомарен: проверки мест, статуса номера, дублей, проживания, ЧС (confirmBlacklisted).
- Выселение атомарно: архив, ЧС, статус номера, запись для выручки, журнал.
- Подселение: номер busy с местами есть в /rooms/available; лишнего гостя сервер не пускает (rooms.full).
- Переименование номера переносит гостей, QR и комментарии.
- Номера чеков — сквозные, без пропусков и дублей при параллельных запросах.
- Журнал пишет события раздела 9 с нужными params.
- Публичные POST /comments и вход ограничены по частоте.
- Дашборд за неделю/месяц/период считает метрики по правилам эндпоинта /dashboard.